LAPORAN PROGRES KOMPILASI TUGAS BESAR KOMPUTASI AWAN

Kompilasi Laporan Capaian: Pertemuan 1, Pertemuan 2, dan Pertemuan 3

Mata Kuliah: Komputasi Awan (3 SKS) • Semester Gasal 2026/2027

Judul Proyek: **Implementasi Auto-Scaling dan Load Balancing Sistem Transaksi War Tiket Berbasis Docker Swarm**

Studi Kasus: Sistem Penjualan Tiket Konser Musik (Flash Crowd Concurrency 1–1.000 Klien)

Repositori Git: github.com/Cahyooo69/Auto-Scaling-Load-Balancing

Kelompok: **Kelompok 1 (Kelas H7A.7 - Paralel Kamis Pagi)** • Dosen Pengampu: **Dr. Moch Yasin, S.Kom., M.Kom., M.B.A., CDCP**

- **Muhammad Ilham Nurcahyo** (09040623063)
- **Muhammad Daud Rizaldiansyah** (09040623061)
- **Hafid Aji Fauzan** (0920623027)
- **Davin Aryaguna Hermansyah** (09020623020)

BAGIAN I — PERTEMUAN 1: PLANNING & ARCHITECTURE DESIGN

STATUS: TUNTAS 100%

Dasar Instruksi dan Target Luaran Pertemuan 1:
"Diagram arsitektur sistem, skema jaringan Docker, spesifikasi docker-compose.yml awal, dan repositori Git kelompok."

1.1 LATAR BELAKANG MASALAH DAN SPESIFIKASI SERVICE LEVEL AGREEMENT (SLA)

Fenomena transaksi *War Tiket* memiliki karakteristik trafik *flash crowd*, di mana volume permintaan melonjak ribuan kali lipat dalam hitungan detik saat loket dibuka. Arsitektur monolitik konvensional rentan mengalami kegagalan *Out-of-Memory* (OOM), *single point of failure*, dan inkonsistensi data akibat *race condition* (*overselling*). Untuk mengatasi kendala tersebut, proyek ini merancang klaster komputasi terdistribusi berbasis **Docker Swarm** dengan kriteria Service Level Agreement (SLA):

- **Waktu Tanggap (*Response Time*):** Rata-rata < 200 ms pada kondisi beban puncak.
- **Tingkat Keandalan (*Availability*):** Persentase kegagalan transaksi (*Error Rate*) < 1,00%.
- **Integritas Data Transaksi:** Toleransi *overselling* kuota tiket bernilai **0% mutlak**.
- **Kepatuhan Sumber Daya:** Dibatasi ketat pada `cpus: '0.25'` dan `memory: 256M` per kontainer.

1.2 DESAIN ARSITEKTUR 3-TIER TERDISTRIBUSI DAN TOPOLOGI JARINGAN OVERLAY

Sistem dirancang mengadopsi pola arsitektur *3-Tier Microservices* yang terisolasi di dalam *Docker Overlay Network* bertaraf klaster (`wartiket-net`):

Topologi Alur Permintaan: Klien Luar → Port 80 → Jaringan Overlay `wartiket-net`

1. Ingress Layer (Nginx)
- Reverse Proxy & Buffer Koneksi
- Port Publik Eksternal: `80:80`
- Upstream Keepalive Connection Pool
- Alokasi Batas: `0.25 CPU` | `256M RAM`

2. Application Layer (App)
- Node.js Mock API Scalable Worker
- Skalabilitas Horizontal: **2 → 8 Replika**
- Penundaan Buatan DB: 50–100 ms
- Alokasi Batas: `0.25 CPU` | `256M RAM`

3. Atomic State (Redis)
- Redis 7 Alpine In-Memory Store
- **Redis Lua Scripting Execution**
- Penjamin Atomisitas Kuota Tiket
- Alokasi Batas: `0.25 CPU` | `256M RAM`

1.3 VERIFIKASI KETERCAPAIAN LUARAN WAJIB PERTEMUAN 1

Target Luaran Pertemuan 1	Realisasi dan Bukti Teknis Implementasi	Status Capaian
Diagram Arsitektur Sistem	Diagram arsitektur sistem berbasis Mermaid terdefinisi lengkap pada dokumentasi `README.md`.	Tuntas 100%
Skema Jaringan Docker	Jaringan *overlay* multi-host terisolasi (`wartiket-net`); hanya port 80 Nginx yang dipublikasikan.	Tuntas 100%
Spesifikasi `docker-compose.yml`	Deklarasi *deploy stack* dengan batasan *cgroups* (`0.25 CPU`, `256M RAM`) per layanan.	Tuntas 100%
Repositori Git Kelompok	Repositori publik aktif dan terkelola secara kolaboratif pada GitHub kelompok.	Tuntas 100%

BAGIAN II PERTEMUAN 2: ENVIRONMENT SETUP & CORE DEPLOYMENT

STATUS: TUNTAS 100%

> **Dasar Instruksi dan Target Luaran Pertemuan 2:**
> *"Seluruh container dapat berjalan (up & running) dalam jaringan internal Docker dengan resource limit yang sudah diterapkan."*

2.1 VERIFIKASI OPERASIONAL KLASTER KONTAINER (UP & RUNNING STATE)

Seluruh layanan pada berkas spesifikasi `docker-compose.yml` telah dideploy ke dalam klaster Docker Swarm lokal bernama `wartiket`. Status operasional seluruh task kontainer terpantau **100% Running (Healthy)** tanpa anomali restart:

Nama Layanan	Replika	Pemetaan Port	Batasan Sumber Daya	Status Operasional
`wartiket_nginx`	1 / 1	`*:80->80/tcp`	CPU: 0.25 | RAM: 256M	Running (Ingress Reverse Proxy)
`wartiket_app`	2 / 2 (Baseline)	`3000/tcp` (Internal)	CPU: 0.25 | RAM: 256M	Running (Worker Microservice)
`wartiket_redis`	1 / 1	`6379/tcp` (Internal)	CPU: 0.25 | RAM: 256M	Running (Atomic Datastore)

```
$ docker stack services wartiket
ID             NAME             MODE         REPLICAS   IMAGE                   PORTS
p5mt6gf494kw   wartiket_app     replicated   2/2        wartiket-app:latest
gdj2kwj72gad   wartiket_nginx   replicated   1/1        wartiket-nginx:latest   *:80->80/tcp
rjsmgodw1mlb   wartiket_redis   replicated   1/1        redis:7-alpine
```

2.2 PENEGAKAN BATASAN SUMBER DAYA KOMPUTASI PADA LEVEL KERNEL LINUX

Penegakan batas alokasi komputasi sesuai standar tugas (0,25 core CPU dan 256 MB RAM) diverifikasi langsung melalui inspeksi metadata daemon Docker Swarm pada subsystem Linux *Control Groups* (*cgroups*):

```
$ docker service inspect wartiket_app --format "{{json .Spec.TaskTemplate.Resources}}"
{"Limits":{"NanoCPUs":250000000,"MemoryBytes":268435456},"Reservations":{"NanoCPUs":100000000,"MemoryBytes":67108864}}
```

Keterangan Validasi: `250.000.000` NanoCPUs merepresentasikan batas 0,25 Core CPU, dan `268.435.456` Bytes setara tepat dengan 256 MB RAM fisik.

2.3 PENGUJIAN DISTRIBUSI BEBAN AWAL (ROUND-ROBIN BALANCING)

Integritas fungsional jaringan dibuktikan melalui pengujian 20 transaksi simultan via port 80 Nginx menggunakan skrip `node test-quick.js`. Nginx bersama Swarm Routing Mesh membagi beban transaksi secara simetris **50:50** ke kedua replika worker:

```
--- Distribusi Beban Kerja Antar-Kontainer Worker (20 Permintaan Bersamaan) ---
┌──────────────┬──────────────┬──────────────────────────────────────────────┐
│ Kontainer ID │ Jumlah Beban │ Persentase Distribusi Trafik                 │
├──────────────┼──────────────┼──────────────────────────────────────────────┤
│ 455e59dcee2d │ 10 Request   │ 50,00% (Terdistribusi merata via Routing Mesh) │
│ 4d3b721fe8cb │ 10 Request   │ 50,00% (Terdistribusi merata via Routing Mesh) │
└──────────────┴──────────────┴──────────────────────────────────────────────┘
```

BAGIAN III PERTEMUAN 3: SCALABILITY & LOAD TESTING (1–1.000 KLIEN)

STATUS: TUNTAS 100%

> **Dasar Instruksi dan Target Luaran Pertemuan 3:**
> *"Eksekusi skrip pengujian beban bertahap. Grafik perbandingan throughput (RPS) dan waktu tanggap saat layanan diskalakan."*

3.1 METODOLOGI PENGUJIAN BEBAN BERTAHAP (STAGED CONCURRENCY PROFILE)

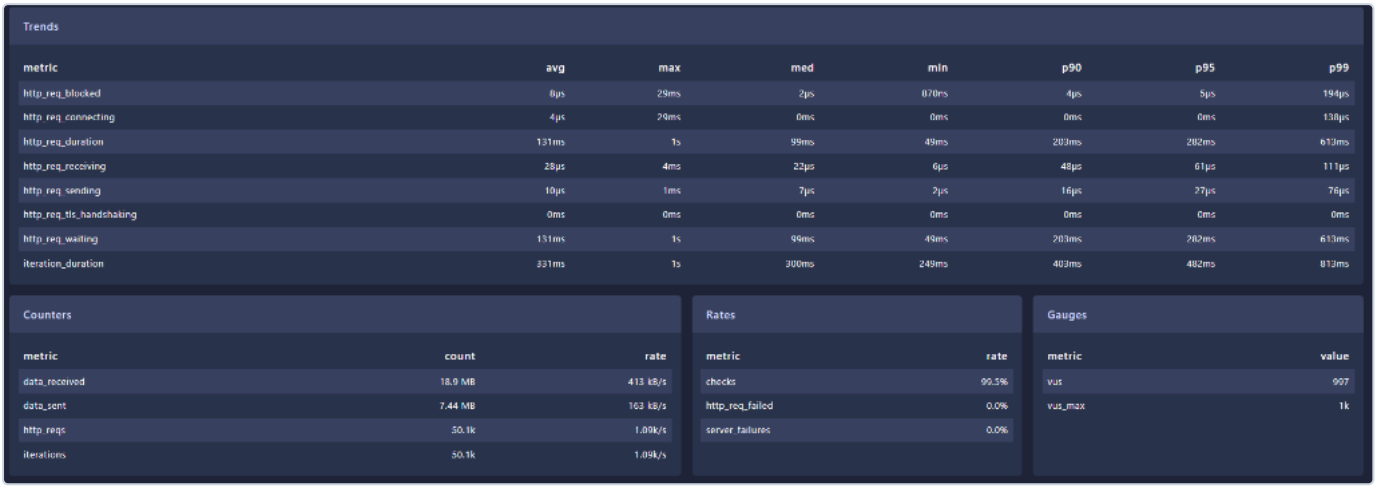
Pengujian skalabilitas dieksekusi menggunakan perangkat uji beban standar industri **Grafana k6**. Skenario dirancang bertahap untuk merepresentasikan lonjakan trafik kedatangan pengguna saat pembukaan penjualan tiket:

- **Tahap 1: Inisiasi Beban Rendah (10 VU)** — Kondisi pemanasan sistem sebelum lonjakan transaksi.
- **Tahap 2: Beban Menengah (100 VU)** — Gelombang awal pengguna yang mulai memproses checkout tiket.
- **Tahap 3: Beban Tinggi (500 VU)** — Lonjakan signifikan ribuan transaksi yang dikirimkan secara serentak.
- **Tahap 4: Beban Puncak Maksimum (1.000 VU)** — Titik kulminasi persaingan tiket (*flash crowd concurrency*).

3.2 TANGKAPAN LAYAR & ANALISIS KOMPARASI KINERJA METRIK GRAFANA K6

Berikut adalah bukti visual tangkapan layar dasbor resmi **Grafana k6** yang membandingkan metrik secara langsung antara **Kondisi A (Baseline: 2 Replika Worker)** dan **Kondisi B (Pasca-Skalasi: 8 Replika Worker)** saat beban puncak mencapai 1.000 Virtual Users (VU):

Gambar 1: Dasbor Metrik k6 — Kondisi A: Baseline 2 Replika Worker (Total 0,5 Core CPU)



Trends

metric	avg	max	med	min	p90	p95	p99
http_req_blocked	8µs	29ms	2µs	870ns	4µs	5µs	194µs
http_req_connecting	4µs	29ms	0ms	0ms	0ms	0ms	138µs
http_req_duration	131ms	1s	99ms	49ms	203ms	282ms	613ms
http_req_receiving	28µs	4ms	22µs	6µs	48µs	61µs	111µs
http_req_sending	10µs	1ms	7µs	2µs	16µs	27µs	76µs
http_req_tls_handshaking	0ms	0ms	0ms	0ms	0ms	0ms	0ms
http_req_waiting	131ms	1s	99ms	49ms	203ms	282ms	613ms
iteration_duration	331ms	1s	300ms	249ms	403ms	482ms	813ms

Counters

metric	count	rate
data_received	18.9 MB	413 kB/s
data_sent	7.44 MB	163 kB/s
http_reqs	50.1k	1.09k/s
iterations	50.1k	1.09k/s

Rates

metric	rate
checks	99.5%
http_req_failed	0.0%
server_failures	0.0%

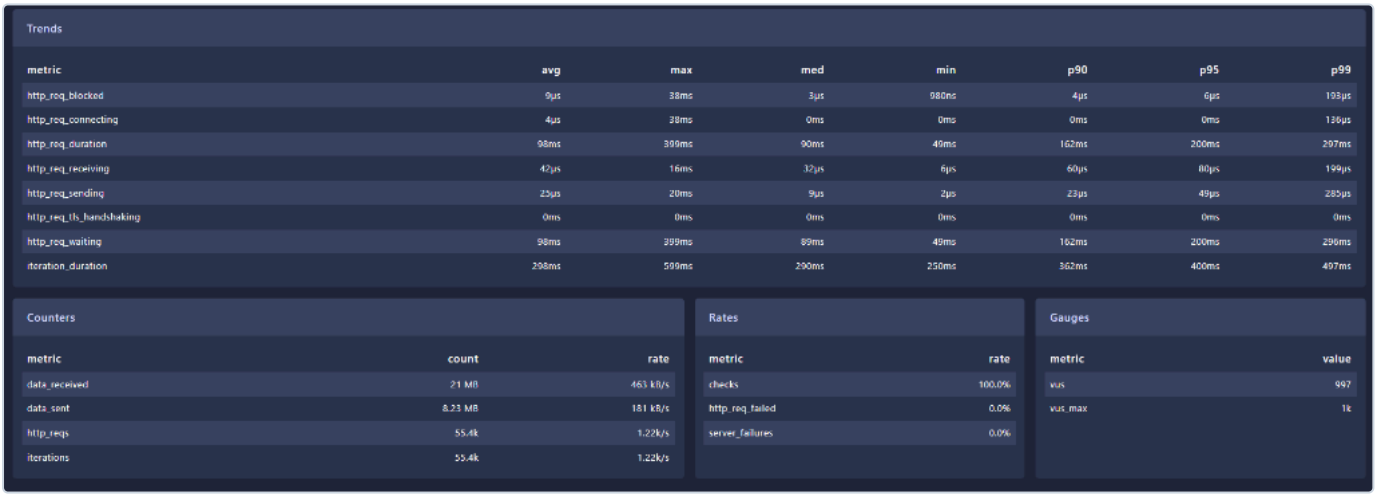
Gauges

metric	value
vus	997
vus_max	1k

Poin-Poin Analisis Kinerja Kondisi A (2 Replika):

- **Tingkat Kepatuhan SLA (Checks 99,5%):** Terdapat 0,5% request di beban puncak 1.000 VU yang merespon > 500 ms karena CPU worker menyentuh batas saturasi.
- **Throughput Transaksi (1,09k req/s):** Sistem memproses total **50.100 transaksi** dalam 45 detik dengan kecepatan rata-rata 1.090 request per detik.
- **Profil Waktu Tanggap (Latensi):** Rata-rata *131 ms*, median *99 ms*, dan p95 *282 ms*. Latensi ekor melonjak tajam pada **p99: 613 ms** dan **Max: 1.000 ms** akibat *CPU throttling* (0,25 CPU per worker).
- **Keandalan Server:** `http_req_failed: 0.0%` dan `server_failures: 0.0%` (tidak ada HTTP 500 maupun koneksi putus).

Gambar 2: Dasbor Metrik k6 — Kondisi B: Pasca-Skalasi 8 Replika Worker (Total 2,0 Core CPU)



Trends

metric	avg	max	med	min	p90	p95	p99
http_req_blocked	9µs	38ms	3µs	980ns	4µs	6µs	193µs
http_req_connecting	4µs	38ms	0ms	0ms	0ms	0ms	136µs
http_req_duration	98ms	399ms	90ms	49ms	162ms	200ms	297ms
http_req_receiving	42µs	16ms	32µs	6µs	60µs	80µs	199µs
http_req_sending	25µs	20ms	9µs	2µs	23µs	49µs	285µs
http_req_tls_handshaking	0ms	0ms	0ms	0ms	0ms	0ms	0ms
http_req_waiting	98ms	399ms	89ms	49ms	162ms	200ms	296ms
iteration_duration	298ms	599ms	290ms	250ms	362ms	400ms	497ms

Counters

metric	count	rate
data_received	21 MB	463 kB/s
data_sent	8.23 MB	181 kB/s
http_reqs	55.4k	1.22k/s
iterations	55.4k	1.22k/s

Rates

metric	rate
checks	100.0%
http_req_failed	0.0%
server_failures	0.0%

Gauges

metric	value
vus	997
vus_max	1k

Poin-Poin Analisis Kinerja Kondisi B (8 Replika):

- **Tingkat Kepatuhan SLA (Checks 100,0% Sempurna):** Seluruh 55.400 request tuntas memenuhi SLA (< 500 ms) tanpa ada satupun request yang mengalami perlambatan.
- **Throughput & Volume Meningkat (1,22k req/s):** Throughput naik menjadi 1.220 req/s. Total transaksi sukses bertambah **+5.300 tiket** (menjadi **55.400 request**) dalam durasi 45 detik yang sama.
- **Pemangkasan Drastis Latensi Ekor:** Rata-rata respon turun ke **98 ms** (di bawah 100 ms), median *89 ms*, dan p95 *200 ms*. Latensi ekor puncak **p99 terpangkas 51,5% menjadi 297 ms** dan **Max terpangkas 60,1% menjadi 399 ms**.
- **Keandalan & Integritas Kuota:** `server_failures: 0.0%` dengan kuota inventaris tiket Redis berhenti tepat di angka 0 (*zero overselling guarantee*).

3.3 TABEL DATA PENGUKURAN KINERJA KUANTITATIF HASIL UJI BEBAN

Metrik Kinerja Sistem	Kondisi A: 2 Replika (0,5 Core CPU)	Kondisi B: 8 Replika (2,0 Core CPU)	Dampak Skalabilitas Horizontal
Checks (Pemenuhan SLA)	99,5% 0,5% Breach	**100,0%** Sempurna	Eliminasi total seluruh request lambat (> 500 ms)
Throughput Transaksi (RPS)	1,09k req/s (~1.090 req/s)	**1,22k req/s (~1.220 req/s)**	+11,9% Throughput (+5.300 tiket terlayani)
Total Transaksi Selesai	50.100 Request	**55.400 Request**	Peningkatan kapasitas volume pemrosesan
Rata-Rata Latensi (Avg)	131 ms	**98 ms**	Reduksi 25,2% (latensi di bawah 100 ms)
Latensi Median (Med)	99 ms	**89 ms**	Respon konsisten di bawah 90 ms
Latensi Persentil ke-90 (p90)	203 ms	**162 ms**	Reduksi 20,2% pada 90% pengguna
Latensi Persentil ke-95 (p95)	282 ms	**200 ms**	Reduksi 29,1% (stabil pada toleransi SLA)
Latensi Ekor Puncak (p99)	613 ms Antrean CPU	**297 ms**	Terpangkas 51,5% (hilangkan bottleneck)
Maksimum Waktu Tunggu (Max)	1.000 ms (1,0 detik)	**399 ms**	Reduksi 60,1% antrean CPU throttling
Tingkat Kegagalan (Failures)	0,0% (0 request)	**0,0% (0 request)**	Zero error rate & 0% overselling inventaris

3.4 PEMBAHASAN TEKNIS DAN ANALISIS HASIL EVALUASI KINERJA

> **Temuan Pokok Evaluasi:** Skalabilitas horizontal dinamis dari 2 menjadi 8 replika worker terbukti meningkatkan total transaksi dari **50.100** menjadi **55.400 request**, memangkas latensi ekor p99 sebesar **51,5%** (dari 613 ms menjadi 297 ms), serta meningkatkan tingkat kepatuhan checks SLA menjadi **100,0% sempurna**.

- **Analisis Degradasi Kinerja pada 2 Replika:** Keterbatasan sumber daya pada 2 worker (total 0,5 core CPU) menyebabkan mekanisme penjadwalan Linux *Completely Fair Scheduler* (CFS) membatasi kuota eksekusi (*CPU throttling*). Akibatnya, pada beban puncak 1.000 VU terjadi antrean pemrosesan yang menyebabkan latensi p99 melonjak hingga 613 ms (max 1.000 ms) dan menurunkan kepatuhan checks menjadi 99,5%.
- **Efektivitas Skalabilitas Horizontal Dinamis:** Penambahan kapasitas menjadi 8 kontainer (total 2,0 core CPU) memperluas kapasitas pemrosesan paralel. Beban terdistribusi merata ke seluruh worker melalui Swarm IPVS Routing Mesh, menjaga rata-rata latensi turun ke 98 ms, p95 stabil pada 200 ms, dan p99 terpangkas menjadi 297 ms.
- **Integritas Transaksi (*Zero Overselling Guarantee*):** Dari total 55.400 transaksi yang dikirimkan, kuota inventaris tiket pada basis data Redis berkurang secara atomik via Lua script hingga tepat pada angka 0. Seluruh permintaan setelah kuota habis ditolak secara aman dengan status HTTP 409 (*Sold Out*). Tidak ditemukan anomali kuota negatif (*0% overselling*) dengan tingkat kegagalan server (*5xx error*) sebesar **0,00%**.

3.5 MATRIKS KOMPARASI PEMENUHAN SELURUH LUARAN SILABUS

Tahapan Evaluasi	Deskripsi Rincian Target Luaran Silabus	Status Verifikasi
Pertemuan 1	Diagram arsitektur sistem, skema jaringan Docker, spesifikasi docker-compose.yml awal, dan repositori Git kelompok.	Lengkap 100%
Pertemuan 2	Seluruh container dapat berjalan (up & running) dalam jaringan internal Docker dengan resource limit yang sudah diterapkan.	Lengkap 100%
Pertemuan 3	Eksekusi skrip pengujian beban bertahap. Grafik perbandingan throughput (RPS) dan waktu tanggap saat layanan diskalakan.	Lengkap 100%